Evaluating an Agent's Architecture

A preparation brief for Simon

Day 2: Agentic AI Architecture / 27 October 2026
Slides 15–19 / approximately six minutes

Summary

An agent can produce a convincing answer while its surrounding system remains fragile: it may retrieve outdated information, repeat an action after a crash, expose private data, or leave no useful record for review. Architecture evaluation asks whether the components and controls around the model support the qualities the application needs.

Your main reading is *AgentArcEval* by Lu et al. [1]. It adapts the Architecture Tradeoff Analysis Method (ATAM) to foundation-model-based agents. Its practical instrument is a **quality scenario**: describe a trigger, the operating conditions, the affected components, the expected response, and a measure of success. The catalogue supplies eleven starting scenarios. Teams adapt and prioritise them for their own domain, analyse architectural support, record risks and trade-offs, and revisit the analysis as the system changes.

The Luna tax-copilot case makes this concrete. Reviewing its architecture exposed missing support for feedback, reuse of previous answers, operational visibility and review of disputed answers. The study shows how scenarios guide architectural improvements. Its numerical targets, including 95% answer accuracy, are **requirements rather than reported benchmark results**.

The sentence to remember: choose the qualities that matter, turn them into checkable scenarios, and examine which architectural mechanisms make the required response possible.

1 Resources consulted and where to focus

This brief follows your actual five-slide section, checked against the current slide source and speaker notes, rather than covering all of Day 2. The presentation handoff assigns you the evaluation topic between Tomás's design patterns and Luca's context engineering.

Primary reading: AgentArcEval. Read Section 1 for the motivation, Section 4 and Figure 2 for the process, Section 5 for the eleven general scenarios, and Sections 6.4–6.5 for Luna's requirements and architectural decisions. The journal reference is *Journal of Systems and Software* 232 (2026), article 112656; the accessible author manuscript is arXiv:2510.21031. This brief uses that manuscript's full text. [1]

Supporting system source: Pi Durable. Earendil's announcement explains the difference between a supervised terminal agent and a durable agent runtime. The sections on crash recovery and tool replay support the availability example on your last slide. This is a system description, not an independent performance evaluation. [2]

Presentation materials. The local files `03-simon-evaluation.json`, `day2_speaker_scripts.py`, `day2_visuals.py`, and the group handoff establish your slide order and examples. Their Pi/Hermes comparison is the presentation team's synthesis. It is not a result of the AgentArcEval study. An accompanying `preparation/README.md` records paths and reading links.

2 Why evaluation changes for agents

Evaluating model outputs remains necessary. It answers questions such as whether a response is correct on a test set. Architecture evaluation asks a related question: *what design choices support that quality, and under what conditions might they fail?* A retrieval component, memory store, permission boundary and recovery mechanism can all affect outcomes even when the underlying model stays the same.

Compound architecture. An agent combines model calls with context assembly, planning, memory, tools, knowledge sources and sometimes other agents. A failure may arise from their interaction. Correct retrieved material can become an incorrect answer; a sound plan can still fail because a tool receives the wrong parameters.

Autonomy and variable execution. A user supplies a broad goal, and the agent chooses intermediate steps while running. Similar requests can produce different tool sequences. Architects therefore need to examine both likely execution paths and the controls that constrain them. An approval gate, execution limit or restricted tool interface makes a boundary concrete.

Evolution. Changing a model, prompt, tool interface or knowledge source can invalidate earlier assumptions. A design review before deployment needs to connect to operational evidence and later reviews. The important question is what changes should trigger another evaluation.

Avoid presenting this as proof that ATAM is obsolete or that traditional methods cannot handle change. AgentArcEval *builds on* ATAM's scenario and trade-off analysis. Its contribution is to focus that analysis on agent-specific components, autonomous decisions, guardrails and continuing operation.

3 How AgentArcEval works

The process starts with stakeholders and the system's purpose. Clarify the agent's goals, intended users and operating context. Review governance needs, translate them into quality and guardrail requirements, and inspect the existing architecture. Architecture diagrams alone are insufficient: reviewers may also need project documents, logs, evaluation evidence and conversations with the team.

At design time, adapt the general scenarios into concrete context scenarios. Rank them by importance, impact, risk and relevance to the goal. Then walk through the architecture: which components respond, what assumptions must hold, and where is support missing?

During operation, monitor changing risks and revisit the scenarios and their priorities. An updated tool, repeated user complaint or newly observed failure can justify another architectural analysis. The paper allows continuous runtime evaluation to be incorporated into an agent, but does not supply a universal automated evaluator that makes all these judgements for a team.

The output is a reasoned account of **trade-offs, risks and recommended changes**. For example, adding a reranker may improve retrieval relevance while increasing latency and cost. Keeping complete execution records may help diagnosis while creating additional privacy and retention obligations. The method makes these tensions explicit; the domain determines the acceptable balance.

3.1 The six-part scenario template

A general scenario is a reusable starting point. A context scenario names the actual users, components and operating conditions of a particular system. Luna's accuracy scenario illustrates all six parts:

Part	Luna accuracy example, shortened from Section 6.4
Source	A tax professional.
Stimulus	A question about capital gains tax when selling a rental property.
Environment	Tax season, when accurate advice is particularly important.
Artefact	Context engine, reasoning/planning, workflow execution, retriever, generator and vector database.
Response	Retrieve relevant current tax regulations and produce advice appropriate to the query's context.
Measure	At least 95% relevant-response accuracy and 95% correct references to applicable regulations.

The two percentages are distinct targets. They do not mean the model is "95% confident", and the paper does not report them as achieved performance. To test such a requirement, a team would also need a defined question set, a reference judgement, a scoring rule and a policy for ambiguous answers. These are practical operational details to add when applying the scenario.

4 The eleven quality attributes

The catalogue provides one starter scenario for each attribute. The measures below condense Section 5; they still need domain-specific definitions and thresholds. On stage, explain a few examples rather than reading every row.

Attribute	Question it raises	Measure proposed in the paper
Accuracy	Does the agent achieve the intended result?	Accuracy relative to user expectations.
Adaptability	Does feedback improve later behaviour?	Successful adaptation rate; adaptation time.
Efficiency	Does reuse avoid unnecessary work?	Response time and cost.
Privacy	Is sensitive information protected?	Percentage of sensitive data protected.
Security	Can the system handle hostile access or attacks?	Detection time and recovery time.
Fairness	Can biased outcomes be detected and mitigated?	Bias detection and mitigation success rates.
Availability	Can execution continue or recover after disruption?	Exception resolution, recovery time, task success and uptime.
Observability	Can operators diagnose what happened?	Log granularity and completeness.
Transparency	Can people understand an outcome's justification?	User understanding and explanation delivery time.
Safety	Does the system avoid harmful behaviour?	Percentage of unsafe events avoided.
Contestability	Can a challenged outcome receive meaningful review?	Resolution time and correction percentage.

Useful distinctions. Observability serves diagnosis; transparency serves understanding; contestability provides a route to challenge and correction. Storing a log does not automatically give a user an understandable explanation, and an explanation alone does not provide an appeal process. Similarly, security concerns hostile interference, while safety also covers harmful actions arising without an attacker.

Luna's review prioritised seven attributes: accuracy, adaptability, efficiency, privacy, observability, transparency and contestability. The four others were not selected for the detailed case analysis. This is a scope choice, not evidence that security, fairness, availability or safety can be ignored in tax software.

5 What the Luna review actually changed

Luna supports tax professionals with recommendations linked to Australian Taxation Office material. The case study is a qualitative architecture walkthrough informed by project stakeholders. Its value is the visible connection between a scenario, a gap in the existing design and a proposed architectural response.

Accuracy versus cost. The review examines retrieval granularity and ranking. Embedding an entire ruling page can mix unrelated material, so the revised design chunks the page by question. It also adds a reranker to improve relevance, accepting extra latency and token cost. The team declines multiple-prompt generation because of the cost of additional model calls. These are architectural decisions justified against a quality goal.

Feedback and efficient reuse. Missing support for adaptation leads to a memory component for feedback and user profiles. For efficiency, the revised planning logic checks for previously answered similar queries. Reuse must also check current legal information: a fast cached response is unhelpful if the underlying rule has changed.

Visibility and review. A cross-cutting AgentOps layer supplies execution records. Responses gain explicit legal sources and model-version information. Feedback storage also tracks the resolution of contested recommendations. Together, these changes support operational diagnosis, understandable outputs and review of disputed advice.

Privacy across the pipeline. The original input desensitiser is insufficient when sensitive information can appear elsewhere. The revised design applies guardrails across components. The lesson is to evaluate the complete data path, including intermediate inputs and outputs.

Do not claim the review demonstrates a measured improvement of a particular size. The case supports the method's practical usefulness in identifying gaps and guiding decisions. It does not establish comparative effectiveness across many applications or certify that the revised system satisfies every target.

6 Using scenarios to discuss Pi, Pi Durable and Hermes

Your final slide applies the catalogue to systems covered elsewhere in the talk. Present this explicitly as **our architectural mapping**, separate from the paper's Luna study. A feature suggests how a scenario might be supported; testing is needed to establish whether the required response actually occurs.

Pi Durable offers the clearest short example. Earendil describes Pi's terminal agent as supervised by a person, while Pi Durable persists task checkpoints and resumes unfinished work after a process restart. Interrupted tools are replayed only when marked safe; other interrupted calls are reported to the model. Submission identifiers prevent duplicate submissions. [2]

That mechanism suggests an availability scenario: interrupt a running task, reopen its persistent storage, resume it, and check recovery time and task completion. Also check whether an external action occurred twice. A deduplicated submission does not by itself guarantee exactly-once effects in every external service. This test proposal is a teaching application of the catalogue, not a reported experiment.

For Pi and Hermes, use the existing slide comparison to point to where mechanisms live: the core, extensions, approvals, logs or memory. Avoid saying that Pi has "no safety" or that Hermes is "safer" merely because a policy module exists. The concrete question is whether the deployed configuration enforces the required boundary under the chosen scenario.

7 Your six-minute speaking route

Slide	Time	Main point to explain
15	60 seconds	Agent execution paths and dependencies change. Evaluate the surrounding mechanisms and guardrails as well as answers.
16	70 seconds	Establish goals and constraints, prioritise scenarios, analyse the architecture, then revisit it using runtime evidence.
17	100 seconds	Walk through Luna's six-part accuracy scenario. Emphasise that 95% is a requirement.
18	60 seconds	The catalogue has eleven starting points. Explain accuracy, safety and contestability; mention Luna's seven priorities.
19	70 seconds	Use crash recovery to connect an attribute to a mechanism and a checkable scenario. Label the system comparison as our synthesis.

Opening: "Tomás gave us a vocabulary for choosing agent designs. My question is how we assess whether those choices support the qualities our application needs."

Closing and handoff: "A scenario connects a quality requirement to the architecture that must support it. Those mechanisms depend on what information the agent can access; Luca will now examine context engineering."

If asked whether this replaces benchmarks: benchmarks and execution tests supply evidence; architecture evaluation explains the design, assumptions and trade-offs behind that evidence. They serve complementary purposes.

If asked whether all eleven are mandatory: the catalogue is a starting point. Select and extend scenarios with stakeholders, while documenting what the evaluation leaves uncovered.

If asked what the paper proves: it demonstrates a structured evaluation on one real tax-copilot architecture. It shows actionable findings, with limited evidence for generalising beyond that case.

References

[1] Q. Lu, D. Zhao, Y. Liu, H. Zhang, L. Zhu, X. Xu, A. Shi, T. Tan and R. Kazman. *AgentArcEval: An Architecture Evaluation Method for Foundation Model Based Agents*. Journal of Systems and Software 232 (2026), 112656. doi:10.1016/j.jss.2025.112656. Accessible manuscript: `https://arxiv.org/html/2510.21031v1`.

[2] Earendil Engineering. *Pi Durable*. 1 October 2026. `https://earendil.com/posts/pi-durable/`. Sections on supervised operation, crash recovery and tools.